

Desenvolvedor(a) em IA para Análise Preditiva [T3]

Mini-Projeto Avaliativo - Módulo 1 - Semana 08

SalesInsight PY — Análise e Visualização de Dados de Vendas com Python

SUMÁRIO

1. CONTEXTUALIZAÇÃO	1
2. DESAFIO	2
3. RESULTADOS ESPERADOS (ENTREGA)	3
4. REQUISITOS DAS TAREFAS	4
4.1. ORGANIZAÇÃO DO PROJETO	4
4.2. REQUISITOS FUNCIONAIS (OBRIGATÓRIOS)	5
RF01 – Criar ou Carregar o Dataset de Vendas	5
RF02 – Inspecionar e Descrever os Dados	7
RF03 – Limpar e Tratar os Dados (datetime e regex)	8
RF04 – Criar Colunas Derivadas com Transformações Condicionais	9
RF05 – Calcular Métricas Agregadas com groupby	9
RF06 – Segmentar Clientes por Nível de Gasto	10
RF07 – Operações Numéricas com NumPy	11
RF08 – Criar Visualizações com Matplotlib e Seaborn	12
RF09 – Organizar o Código em Funções Reutilizáveis e em uma Classe	13
RF10 – Exportar Resultados em CSV e JSON	15
RF11 – Executar o Fluxo Completo (Ponto de Entrada)	15
4.3. REQUISITOS BÔNUS (OPCIONAIS)	16
4.4. REQUISITOS TÉCNICOS	16
4.5. ORGANIZAÇÃO DO KANBAN	19
4.6. VERSIONAMENTO COM GITHUB	20
4.7. GRAVAÇÃO DE VÍDEO	21
5. CRITÉRIOS DE AVALIAÇÃO	21
6. CHECKLIST FINAL DE ENTREGA	23
7. MODELO DE README.md (SUGESTÃO)	24

1. CONTEXTUALIZAÇÃO

O mercado de Inteligência Artificial e Ciência de Dados exige que você saiba demonstrar raciocínio lógico, organização, capacidade de resolver problemas com dados reais e boa comunicação técnica. Mesmo antes de construir modelos preditivos, já é possível

entregar um projeto completo e útil para o seu portfólio usando Python, Pandas, NumPy e visualização de dados.

Neste mini-projeto você desenvolverá um analisador de dados de vendas organizado em funções reutilizáveis e em uma classe, aproximando-se de uma situação real de trabalho: carregar, limpar, transformar, agregar e visualizar dados para gerar insights acionáveis.

O projeto consolida **exclusivamente** os conteúdos já trabalhados até a Semana 08 do Módulo 01:

- **Semanas 01–02:** fundamentos da análise preditiva; conceitos de dados, features e variável-alvo.
- **Semana 03:** lógica de programação com Python — variáveis, tipos de dados, operadores aritméticos, lógicos e relacionais, condicionais (if/elif/else) e repetição (for/while).
- **Semana 04:** versionamento com Git e GitHub (branches, GitFlow); listas, tuplas, dicionários e estruturas compostas.
- **Semana 05:** leitura e escrita de arquivos (CSV e JSON); módulo datetime; expressões regulares básicas; funções, parâmetros, retorno, funções lambda, módulos e importação.
- **Semana 06:** Pandas — Series e DataFrames, leitura de dados, filtros e seleções, groupby, merge e pivot; NumPy — arrays.
- **Semana 07:** NumPy — operações vetorizadas e broadcasting; transformações — aplicação de funções em colunas, transformações condicionais, junção e concatenação, tratamento de dados “sujos”.
- **Semana 08:** visualização com Matplotlib e Seaborn — gráficos de linha, barra e dispersão, customização, subplots e layouts, exportação de gráficos; preparação para pipelines — funções para transformações reutilizáveis e introdução a classes para organização de código.

Importante: conteúdos das semanas seguintes **não** são exigidos neste mini-projeto. Não é necessário (e não será cobrado) imputação estatística de valores ausentes, detecção e remoção de outliers, normalização/padronização como etapa de pré-processamento (Semanas 09–11), estatística inferencial e correlação (Semana 12) ou qualquer modelo preditivo, incluindo regressão linear (Semana 13).

O projeto também reforça sua organização profissional com o uso do GitHub Flow, com branches descritivas, commits de mensagem clara e um histórico que reflita a evolução lógica do trabalho.

2. DESAFIO

Você acaba de ser contratado como Analista de Dados Júnior em uma empresa de varejo que possui um histórico de vendas em formato CSV e precisa de um relatório analítico para a reunião trimestral da diretoria. O time de negócios quer entender:

- Como as vendas se comportam ao longo do tempo (por mês e por trimestre);
- Quais produtos e categorias geram mais receita?

- Quais regiões têm melhor desempenho?
- Quais clientes são mais valiosos (segmentação simples por faixa de gasto)?
- Existe relação entre a quantidade vendida e a receita gerada por transação?

Você deverá desenvolver um mini-projeto em Python chamado:

SalesInsight PY — Análise e Visualização de Dados de Vendas

O objetivo é organizar um fluxo de análise de dados que **carregue, limpe, transforme, agregue e visualize** um dataset de vendas, gerando insights relevantes e exportando os resultados. A aplicação deverá:

- Carregar e inspecionar os dados brutos de um arquivo CSV;
- Limpar e tratar dados inconsistentes (valores nulos, datas inválidas, espaços extras e ruído em textos);
- Criar colunas derivadas (receita por linha, mês, trimestre, ano e faixa de valor);
- Calcular métricas agregadas por mês, produto, categoria e região;
- Segmentar clientes por nível de gasto (Bronze, Prata, Ouro);
- Gerar visualizações claras e informativas, incluindo uma figura com subplots;
- Exportar um relatório resumido em CSV, estatísticas em JSON e os gráficos em PNG;
- Organizar todo o fluxo em funções reutilizáveis e em ao menos uma classe.

Sobre a palavra “pipeline”

Neste projeto, **pipeline** significa apenas uma sequência organizada e reutilizável de funções e métodos que transformam os dados de ponta a ponta. Não se trata do objeto `Pipeline` do Scikit-learn, que será estudado no Módulo 02.

Maneira de execução

- **Google Colab** (recomendado para a disciplina): faça upload do arquivo `.py` ou `.ipynb` e execute as células;
- **VS Code** com a extensão Python, executando o arquivo `.py` localmente pelo terminal;
- **Terminal / Prompt de Comando:** `python salesinsight.py` (requer Python e as bibliotecas instaladas via pip).

O Google Colab já possui Pandas, NumPy, Matplotlib e Seaborn instalados. Para execução local, instale as dependências com `pip install pandas numpy matplotlib seaborn`.

3. RESULTADOS ESPERADOS (ENTREGA)

Ao final do mini-projeto, o estudante ou a squad (grupo de até 03 pessoas) deverá entregar:

- Um repositório **público** no GitHub com o projeto;
- Um arquivo Python principal chamado `salesinsight.py` (e/ou um notebook `salesinsight.ipynb`);

- O arquivo do dataset utilizado (`vendas.csv`) ou as instruções para gerá-lo, caso seja gerado sinteticamente pelo próprio código;
- Um arquivo `README.md` explicando o projeto;
- Um histórico de commits no GitHub;
- Um vídeo de até 5 minutos demonstrando o funcionamento;
- Os links submetidos no AVA.

Entrega no AVA

Item	Informação
Tarefa no AVA	Módulo 01 – Mini-Projeto Avaliativo
Avaliação	M1.1 — 25% da nota do Módulo 01
Nota	0 (zero) a 10 (dez)
Formato	Individual ou em squad de até 03 pessoas
Data de liberação	14/08/2026, às 22h
Data de entrega	24/08/2026, até às 22h

Links obrigatórios na submissão

- Link do repositório público no GitHub;
- Link do vídeo de demonstração (Google Drive ou YouTube).

4. REQUISITOS DAS TAREFAS

4.1. ORGANIZAÇÃO DO PROJETO

A estrutura mínima do projeto deverá ser:

```
salesinsight-py/  
|  
|-- salesinsight.py  
|-- vendas.csv  
|-- README.md
```

Squads (ou quem quiser se organizar melhor) podem adotar a seguinte estrutura:

```
salesinsight-py/  
|  
|-- README.md  
|-- salesinsight.py  
|-- vendas.csv  
|-- outputs/  
|   |-- metricas_por_mes.csv
```

```
| |-- segmentacao_clientes.csv  
| |-- estatisticas_gerais.json  
| |-- graficos/  
|     |-- receita_por_mes.png  
|     |-- top_produtos.png  
|     |-- quantidade_vs_receita.png  
|     |-- painel_resumo.png  
|-- planejamento/  
    |-- tarefas-kanban.md
```

Não é obrigatório criar todas as pastas. O foco é demonstrar domínio dos conceitos de Python, Pandas, NumPy e visualização estudados **até a Semana 08**.

4.2. REQUISITOS FUNCIONAIS (OBRIGATÓRIOS)

São **11 requisitos funcionais obrigatórios**. Os exemplos de código apresentados são referências de apoio: você deve compreender, adaptar e ser capaz de explicar cada trecho que entregar. Nos requisitos avaliados diretamente, são fornecidas apenas as assinaturas e orientações — a implementação é sua.

RF01 – Criar ou Carregar o Dataset de Vendas

O projeto deverá ter um dataset de vendas. Você pode **gerar o dataset sinteticamente** no próprio código (recomendado, pois garante a reprodução por qualquer pessoa) ou utilizar um CSV real.

Opção A – Gerar o dataset no código (recomendado)

O gerador abaixo cria propositalmente dados “sujos”, que servirão de matéria-prima para o requisito de limpeza. Use-o como está:

```
Python  
import pandas as pd  
import numpy as np  
import random  
from datetime import datetime, timedelta  
  
def gerar_dataset_vendas(n_registros=200, seed=42):  
    """Gera um dataset sintetico de vendas com dados sujos."""  
    random.seed(seed)  
    np.random.seed(seed)  
  
    produtos = ["Notebook", "Smartphone", "Tablet", "Monitor",  
                "Teclado", "Mouse", "Headset"]  
    categorias = {"Notebook": "Computadores", "Smartphone": "Celulares",  
                 "Tablet": "Celulares", "Monitor": "Computadores",  
                 "Teclado": "Perifericos", "Mouse": "Perifericos",  
                 "Headset": "Perifericos"}  
    precos = {"Notebook": 3500, "Smartphone": 2200, "Tablet": 1800,  
             "Monitor": 1200, "Teclado": 250, "Mouse": 120,  
             "Headset": 350}  
    regioes = ["Sudeste", "Sul", "Nordeste", "Centro-Oeste", "Norte"]
```

```
data_inicio = datetime(2025, 1, 1)
dados = []

for i in range(n_registros):
    produto = random.choice(produtos)
    categoria = categorias[produto]
    quantidade = random.randint(1, 10)
    preco = round(preco[produto] * random.uniform(0.85, 1.15), 2)
    data = data_inicio + timedelta(days=random.randint(0, 364))
    data_txt = data.strftime("%Y-%m-%d")
    cliente = f"Cliente_{random.randint(1, 50):03d}"

    # --- sujeira proposital para a etapa de limpeza ---
    if random.random() < 0.05:
        quantidade = None                # valor nulo
    if random.random() < 0.04:
        preco = None                     # valor nulo
    if random.random() < 0.06:
        produto = " " + produto + " "   # espacos extras
    if random.random() < 0.03:
        data_txt = "DATA INVALIDA"       # data invalida
    if random.random() < 0.10:
        cliente = random.choice([        # ruído no nome
            cliente.upper().replace("_", "-"),
            cliente + "!!",
            " " + cliente,
            cliente.replace("Cliente_", "cliente#"),
        ])

    dados.append({
        "id_venda": i + 1,
        "data_venda": data_txt,
        "cliente": cliente,
        "produto": produto,
        "categoria": categoria,
        "regiao": random.choice(regioes),
        "quantidade": quantidade,
        "preco_unitario": preco,
    })

return pd.DataFrame(dados)

# Gerar e salvar o CSV bruto
df_bruto = gerar_dataset_vendas()
df_bruto.to_csv("vendas.csv", index=False)
print(f"Dataset gerado com {len(df_bruto)} registros.")
print(df_bruto.head())
```

Opção B – Usar um CSV externo

Você pode baixar um dataset público (Kaggle, dados.gov.br etc.) e adaptá-lo, desde que contenha ao menos colunas de **data**, **produto**, **categoria**, **quantidade**, **preço unitário** e **região**, e que exista alguma inconsistência real a ser tratada.

RF02 – Inspecionar e Descrever os Dados

O projeto deverá inspecionar o dataset carregado, exibindo suas informações estruturais no console: formato (`.shape`), colunas, tipos de dados (`.dtypes`), contagem de valores nulos por coluna (`.isnull().sum()`) e os primeiros registros (`.head()`).

Nesta etapa o objetivo é **descrever a estrutura** do DataFrame, não interpretar estatísticas. A leitura estatística dos dados será trabalhada na Semana 12.

Exemplo de saída esperada no console

```
=== INSPECAO INICIAL DO DATASET ===
Shape: (200, 8)
Colunas: ['id_venda', 'data_venda', 'cliente', 'produto',
          'categoria', 'regiao', 'quantidade', 'preco_unitario']

Tipos de dados:
  id_venda          int64
  data_venda        object
  quantidade        float64
  ...

Valores nulos por coluna:
  quantidade      10
  preco_unitario   8
  ...
```

Exemplo de implementação

```
Python
def inspecionar_dados(df):
    """Exibe as informacoes estruturais do DataFrame."""
    print("\n=== INSPECAO INICIAL DO DATASET ===")
    print(f"Shape: {df.shape}")
    print(f"Colunas: {list(df.columns)}")
    print(f"Tipos de dados:\n{df.dtypes}")
    print(f"Valores nulos por coluna:\n{df.isnull().sum()}")
    print(f"Primeiros registros:\n{df.head()}")
    return df
```

RF03 – Limpar e Tratar os Dados (datetime e regex)

Implemente uma função de limpeza que trate, no mínimo:

- **Espaços extras** em colunas de texto, com `.str.strip()`;
- **Datas inválidas:** converter `data_venda` com `pd.to_datetime(..., errors="coerce")` e **remover** as linhas que não puderam ser convertidas;
- **Valores nulos** nas colunas críticas `quantidade` e `preco_unitario`, com `.dropna(subset=[...])`;
- **Tipos de dados:** garantir `int` para `quantidade` e `float` para `preco_unitario`;

- **Ruído nos nomes de cliente**, usando o módulo `re`: remover caracteres não alfanuméricos, padronizar o formato para `Cliente_NNN` e sinalizar os registros fora do padrão;
- **Relatório de limpeza**: exibir no console quantos registros entraram, quantos foram removidos (por motivo) e quantos permaneceram.

Trabalhe apenas com **remoção** de registros inválidos e padronização de texto. Estratégias de imputação de nulos e tratamento de outliers pertencem à Semana 10 e **não** devem ser usadas aqui.

Assinatura sugerida

```
Python
import re

def limpar_dados(df):
    """
    Limpa e trata o DataFrame de vendas.
    Retorna: (df_limpo, relatorio), onde relatorio e um dicionario
    com as contagens de registros iniciais, removidos e finais.
    """
    # 1. remover espacos extras nas colunas de texto
    # 2. converter data_venda e descartar datas invalidas
    # 3. descartar nulos em quantidade e preco_unitario
    # 4. ajustar os tipos numericos
    # 5. padronizar o nome do cliente com re.sub() e re.compile()
    # 6. montar e imprimir o relatorio de limpeza
    ...
```

Trechos de apoio

```
Python
# conversao de datas marcando os invalidos como NaT
df["data_venda"] = pd.to_datetime(df["data_venda"], errors="coerce")

# limpeza de texto com expressao regular
df["cliente"] = df["cliente"].apply(
    lambda s: re.sub(r"^[A-Za-z0-9_]", "", str(s).strip())
)

# validacao de formato com padrao compilado
padrao_cliente = re.compile(r"^Cliente_\d{3}$", flags=re.IGNORECASE)
```

RF04 – Criar Colunas Derivadas com Transformações Condicionais

A partir do dataset limpo, crie no mínimo as seguintes colunas:

Coluna	Como obter
receita_total	quantidade × preco_unitario (operação vetorizada entre Series)

mes	Componente de mês extraído da data com <code>.dt.month</code>
mes_nome	Nome do mês em português, a partir de um dicionário de mapeamento
trimestre	Trimestre no formato Q1–Q4, a partir de <code>.dt.quarter</code>
ano	Ano extraído da data com <code>.dt.year</code>
faixa_receita_item	Classificação condicional da receita da linha com <code>np.select</code>

Atenção: `.dt.strftime("%B")` retorna o nome do mês **em inglês** no ambiente padrão do Google Colab. Use um dicionário de mapeamento (`{1: "Janeiro", 2: "Fevereiro", ...}`) para obter os nomes em português.

Trecho de apoio – transformação condicional vetorizada

```
Python
condicoes = [
    df["receita_total"] < 500,
    (df["receita_total"] >= 500) & (df["receita_total"] < 5000),
    df["receita_total"] >= 5000,
]
faixas = ["Baixo Valor", "Medio Valor", "Alto Valor"]
df["faixa_receita_item"] = np.select(condicoes, faixas,
                                     default="Nao Classificado")
```

RF05 – Calcular Métricas Agregadas com `groupby`

O projeto deverá calcular e exibir métricas agrupadas em, no mínimo, quatro dimensões:

- Receita total, quantidade vendida e número de vendas **por mês**;
- Receita total **por produto** (Top 5, em ordem decrescente);
- Receita total **por categoria**;
- Receita total e ticket médio **por região**.

As métricas devem ser retornadas em uma estrutura composta (por exemplo, um dicionário de DataFrames) e exibidas no console de forma legível.

Assinatura sugerida

```
Python
def calcular_metricas(df):
    """
    Calcula as metricas agregadas do dataset.
    Retorna um dicionario no formato {nome_da_metrica: DataFrame}.
    Chaves minimas: por_mes, top_produtos, por_categoria, por_regiao.
    """
```

```
metricas = {}  
# use df.groupby(...) com .agg(), .sum(), .mean(),  
# .sort_values() e .reset_index() para deixar cada  
# resultado em formato tabular  
...  
return metricas
```

Exemplo de saída esperada no console

```
=== POR MES ===  
mes  receita_total  quantidade  n_vendas  
1      45230.50         312         18  
2      38910.00         289         15  
...  
  
=== TOP PRODUTOS ===  
produto  receita_total  
Notebook    125430.00  
Smartphone   98320.50  
...
```

RF06 – Segmentar Clientes por Nível de Gasto

Agrupe os dados por cliente, calcule o total gasto por cada um e classifique-os em segmentos. A classificação deve usar **ao menos uma função lambda** aplicada com `.apply()`.

Gasto total acumulado	Segmento
Abaixo de R\$ 5.000,00	Bronze
De R\$ 5.000,00 a R\$ 15.000,00	Prata
Acima de R\$ 15.000,00	Ouro

Exiba os 10 maiores clientes e a distribuição de clientes por segmento (`.value_counts()`).

Assinatura sugerida

```
Python  
def segmentar_clientes(df):  
    """  
    Agrupa por cliente, soma a receita e classifica em  
    Bronze / Prata / Ouro.  
    Retorna um DataFrame com: cliente, total_gasto, segmento.  
    """  
    ...
```

RF07 – Operações Numéricas com NumPy

O projeto deverá usar NumPy diretamente sobre os dados, demonstrando:

- **Conversão** de uma coluna do DataFrame para array NumPy (`.to_numpy()`);
- **Operações vetorizadas**, sem nenhum laço `for` sobre o array;
- **Broadcasting**: uma operação aritmética entre um array e um valor escalar (ou entre arrays de formatos compatíveis);
- **Filtragem booleana** de array (ex.: quantas vendas ficaram acima do valor médio);
- **Ao menos três funções NumPy** de agregação — `np.mean`, `np.median`, `np.std`, `np.sum`, `np.min`, `np.max`.

Aqui o objetivo é demonstrar o **domínio da API do NumPy** — arrays, vetorização e broadcasting — e não realizar análise estatística. O escalonamento de valores para o intervalo 0-1 pedido abaixo é um exercício de aritmética vetorizada; a normalização como etapa de pré-processamento de modelos será estudada na Semana 10.

Assinatura sugerida

```
Python
def calcular_estatisticas_numpy(df):
    """
    Aplica operacoes NumPy sobre a coluna receita_total.
    Retorna um dicionario com os valores agregados calculados.
    """
    receitas = df["receita_total"].to_numpy()

    # agregacoes com np.mean / np.median / np.std / np.sum
    # broadcasting: escalonar o array para o intervalo 0-1
    # (receitas - receitas.min()) / (receitas.max() - receitas.min())
    # filtragem booleana: receitas[receitas > media]
    ...
```

Observação técnica: `np.std()` usa por padrão `ddof=0` e `pandas.Series.std()` usa `ddof=1`. Os dois resultados serão ligeiramente diferentes — isso é esperado, e vale mencioná-lo no README ou no vídeo.

RF08 – Criar Visualizações com Matplotlib e Seaborn

Este é o requisito central da Semana 08. O projeto deverá gerar **quatro figuras**, todas exportadas em PNG com `plt.savefig()`:

Nº	Figura	Tipo	Conteúdo
1	receita_por_mes.png	Linha	Receita total por mês ao longo do período
2	top_produtos.png	Barra	Top 5 produtos (ou categorias) por receita

3	quantidade_vs_receita.png	Dispersão	quantidade × receita_total, colorido por categoria
4	painel_resumo.png	Subplots	Figura 2×2 combinando as visões acima e a receita por região

Requisitos de customização — obrigatórios em todas as figuras:

- Título da figura e rótulos nos dois eixos;
- Legenda, quando houver mais de uma série ou agrupamento por cor;
- Tamanho de figura definido explicitamente (`figsize` ou `plt.rcParams`);
- Paleta ou cores escolhidas deliberadamente (não os padrões da biblioteca);
- Ajuste de layout com `plt.tight_layout()` e exportação com `dpi` de no mínimo 100;
- Na figura de subplots: uso de `plt.subplots(2, 2)`, título geral com `fig.suptitle()` e eixos acessados individualmente.

Trechos de apoio

```
Python
import matplotlib.pyplot as plt
import seaborn as sns
import os

# configuracao visual global
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.figsize"] = (12, 6)
plt.rcParams["axes.titlesize"] = 14

# --- figura simples: linha ---
fig, ax = plt.subplots()
ax.plot(por_mes["mes"], por_mes["receita_total"],
        marker="o", linewidth=2)
ax.set_title("Receita Total por Mes")
ax.set_xlabel("Mes")
ax.set_ylabel("Receita Total (R$)")
plt.tight_layout()
os.makedirs("outputs/graficos", exist_ok=True)
plt.savefig("outputs/graficos/receita_por_mes.png", dpi=150)
plt.close()

# --- barras com Seaborn (atencao a assinatura atual) ---
fig, ax = plt.subplots()
sns.barplot(data=top_produtos, y="produto", x="receita_total",
            hue="produto", legend=False, palette="Blues_d", ax=ax)

# --- figura com subplots 2x2 ---
fig, axes = plt.subplots(2, 2, figsize=(14, 9))
axes[0, 0].plot(...)      # linha
axes[0, 1].bar(...)       # barras
axes[1, 0].scatter(...)   # dispersao
axes[1, 1].bar(...)       # receita por regioao
```

```
fig.suptitle("SalesInsight PY - Painei Resumo", fontsize=16)
plt.tight_layout()
plt.savefig("outputs/graficos/painei_resumo.png", dpi=150)
plt.close()
```

Nas versões atuais do Seaborn, passar `palette` sem `hue` gera aviso de descontinuação (e erro em versões mais novas). Sempre que usar `palette` em `barplot` ou `boxplot`, informe também `hue` e `legend=False`, como no exemplo acima.

RF09 – Organizar o Código em Funções Reutilizáveis e em uma Classe

Este requisito atende ao tema “preparação para pipelines” da Semana 08 e tem duas partes.

Parte A – Funções reutilizáveis e função de ordem superior

Todo o fluxo deve estar organizado em funções com parâmetros, retorno e docstring. Além disso, implemente **uma função que receba outra função como argumento** e a aplique a uma coluna do DataFrame, demonstrando o conceito de função de ordem superior.

```
Python
def processar_coluna(df, coluna, funcao_transformacao, nome_saida=None):
    """
    Aplica uma funcao de transformacao a uma coluna do DataFrame.
    Demonstra o uso de funcoes como argumento.
    """
    nome_saida = nome_saida or f"{coluna}_transformado"
    df[nome_saida] = df[coluna].apply(funcao_transformacao)
    return df

# uso com funcoes lambda em contextos distintos
df = processar_coluna(df, "receita_total",
                      lambda x: round(x / 1000, 2),
                      nome_saida="receita_em_milhares")
df = processar_coluna(df, "quantidade",
                      lambda q: "Alto Volume" if q > 5 else "Baixo Volume",
                      nome_saida="perfil_volume")
```

Parte B – Uma classe para organizar o fluxo

Crie **uma classe** que encapsule o fluxo de análise. A classe deve ter, no mínimo:

- Um construtor `__init__` que receba o caminho do arquivo e inicialize os atributos;
- Atributos de instância para guardar o estado (DataFrame bruto, DataFrame limpo, métricas, relatório de limpeza);
- Métodos de instância que utilizem `self` e reaproveitem as funções já criadas (carregar, limpar, transformar, analisar, visualizar, resumo).

Escopo desta semana: **introdução a classes** — construtor, atributos e métodos. **Não** é exigido herança, `super()`, métodos estáticos, propriedades ou encadeamento fluente de

métodos. Quem quiser explorar herança encontrará essa possibilidade na seção 4.3 (Requisitos Bônus), sem impacto na nota.

Estrutura sugerida

Python

```
class AnalisadorDeVendas:
    """Encapsula o fluxo de análise dos dados de vendas."""

    def __init__(self, caminho_arquivo):
        self.caminho_arquivo = caminho_arquivo
        self.df_bruto = None
        self.df_limpo = None
        self.metricas = {}
        self.clientes = None
        self.relatorio_limpeza = {}

    def carregar(self):
        """Le o CSV e guarda o DataFrame bruto."""
        self.df_bruto = pd.read_csv(self.caminho_arquivo)
        print(f"[Analisador] {len(self.df_bruto)} registros lidos.")

    def limpar(self):
        """Limpa os dados reaproveitando limpar_dados()."""
        self.df_limpo, self.relatorio_limpeza = limpar_dados(
            self.df_bruto.copy())

    def transformar(self):
        """Cria as colunas derivadas."""
        self.df_limpo = criar_colunas_derivadas(self.df_limpo)

    def analisar(self):
        """Calcula metricas, segmentacao e operacoes NumPy."""
        ...

    def visualizar(self):
        """Gera e exporta as figuras."""
        ...

    def resumo(self):
        """Imprime um resumo executivo do que foi processado."""
        ...
```

RF10 – Exportar Resultados em CSV e JSON

O projeto deverá demonstrar **leitura e escrita** de arquivos em dois formatos:

- Escrever as métricas por mês em CSV com `.to_csv()`;
- Escrever a segmentação de clientes em CSV;
- Escrever as estatísticas gerais em JSON com `json.dump()`;
- **Ler de volta** o JSON gravado com `json.load()` e exibir o conteúdo, comprovando a operação.

Python

```
import json
import os

def exportar_resultados(metricas, clientes, estatisticas):
    """Exporta os resultados do projeto em CSV e JSON."""
    os.makedirs("outputs", exist_ok=True)

    metricas["por_mes"].to_csv("outputs/metricas_por_mes.csv",
                              index=False, encoding="utf-8-sig")
    clientes.to_csv("outputs/segmentacao_clientes.csv",
                   index=False, encoding="utf-8-sig")

    serializavel = {k: round(float(v), 2)
                    for k, v in estatisticas.items()}
    caminho = "outputs/estatisticas_gerais.json"
    with open(caminho, "w", encoding="utf-8") as f:
        json.dump(serializavel, f, indent=4, ensure_ascii=False)

    # leitura de volta para confirmar a escrita
    with open(caminho, "r", encoding="utf-8") as f:
        conferencia = json.load(f)
    print(f"JSON gravado e lido: {conferencia}")
```

RF11 – Executar o Fluxo Completo (Ponto de Entrada)

O projeto deverá ter uma função `main()` e o bloco `if __name__ == "__main__":`, executando todas as etapas na ordem correta, de ponta a ponta, sem erros.

Atenção à ordem: **toda** a limpeza (incluindo a padronização com expressões regulares) deve ocorrer **antes** das transformações, das agregações e dos gráficos. Não faça limpeza depois de gerar as visualizações.

Python

```
def main():
    """Executa o fluxo completo do SalesInsight PY."""
    print("=" * 60)
    print("  SALESINSIGHT PY - Analise de Dados de Vendas")
    print("=" * 60)

    # Etapa 0 - garantir a existencia do dataset
    if not os.path.exists("vendas.csv"):
        gerar_dataset_vendas().to_csv("vendas.csv", index=False)

    # Etapas 1 a 6 - fluxo pela classe
    analisador = AnalisadorDeVendas("vendas.csv")
    analisador.carregar()
    inspecionar_dados(analisador.df_bruto)
    analisador.limpar()
    analisador.transformar()
    analisador.analisar()
```

```
analizador.visualizar()
analizador.resumo()

print("\n[CONCLUIDO] Fluxo finalizado com sucesso.")

if __name__ == "__main__":
    main()
```

4.3. REQUISITOS BÔNUS (OPCIONAIS)

Os itens abaixo **não são obrigatórios e não afetam a nota**. São caminhos de aprofundamento para quem terminar antes do prazo e quiser enriquecer o portfólio — podem ser destacados no README e no vídeo como diferencial.

- **B01 – Herança:** criar uma subclasse (ex.: `AnalizadorComProjecao`) que herde de `AnalizadorDeVendas` usando `super()`. *Conteúdo além do Módulo 01.*
- **B02 – Projeção simples de tendência:** estimar a receita dos próximos meses por média móvel dos últimos meses. *Modelos preditivos serão estudados na Semana 13.*
- **B03 – merge ou pivot:** enriquecer o dataset com uma tabela auxiliar (ex.: metas por região) ou montar uma tabela cruzada mês × categoria.
- **B04 – Gráficos adicionais:** histograma, boxplot, mapa de calor ou barras agrupadas, além das quatro figuras obrigatórias.
- **B05 – Percentis:** calcular `np.percentile` para os quartis da receita. *A interpretação de quartis será trabalhada na Semana 12.*

4.4. REQUISITOS TÉCNICOS

A tabela a seguir relaciona cada conteúdo do módulo à forma de demonstrá-lo no projeto.

Conteúdo	Semana	Obrig.	Como demonstrar
Kanban	—	Sim	Quadro com as tarefas do projeto
Google Colab ou VS Code	03/04	Sim	Usar como ambiente de desenvolvimento
Versionamento com Git	04	Sim	Commits no repositório
GitHub (repositório público)	04	Sim	Link do repositório no AVA
Branches / GitFlow simplificado	04	Sim	Branches com nomes descritivos
GitHub Desktop	04	Opcional	Pode ser usado para commits e push

Lógica de programação	03	Sim	Cálculos, condicionais e repetições
Tipos de dados	03	Sim	int, float, str, bool, list, dict
Operadores aritméticos, lógicos e relacionais	03	Sim	Cálculo de receita, filtros e comparações
if / elif / else	03	Sim	Classificação de faixas e segmentos
for / while	03	Sim	Geração do dataset e exibição de resultados
Listas, tuplas, dicionários e estruturas compostas	04	Sim	Dataset, dicionário de métricas, relatório de limpeza
Leitura e escrita de CSV	05	Sim	<code>pd.read_csv()</code> e <code>.to_csv()</code>
Leitura e escrita de JSON	05	Sim	<code>json.dump()</code> e <code>json.load()</code>
Módulo datetime	05	Sim	Geração de datas e extração de mês, trimestre e ano
Expressões regulares (re)	05	Sim	<code>re.sub()</code> e <code>re.compile()</code> na limpeza de textos
Funções com parâmetros, retorno e docstring	05	Sim	Todas as funções do fluxo
Funções lambda	05	Sim	Ao menos 2 usos em contextos distintos
Função que recebe função como argumento	05	Sim	<code>processar_coluna()</code> ou equivalente
Módulos e importação	05	Sim	<code>import pandas, numpy, json, re, os</code>
Pandas — Series e DataFrames	06	Sim	Estrutura principal de dados do projeto
Pandas — leitura de fontes de dados	06	Sim	<code>read_csv()</code> com tratamento posterior
Pandas — filtros e seleções	06	Sim	Filtros com condições booleanas
Pandas — groupby	06	Sim	Agregações por mês, produto, categoria e região
Pandas — merge ou pivot	06	Opcional	Tabela auxiliar ou tabela cruzada (bônus B03)

NumPy — arrays	06	Sim	<code>.to_numpy()</code> sobre colunas do DataFrame
NumPy — operações vetorizadas	07	Sim	Cálculos sem laços sobre arrays
NumPy — broadcasting	07	Sim	Operação entre array e escalar
Transformações condicionais (<code>np.select</code>)	07	Sim	Coluna <code>faixa_receita_item</code>
Aplicação de funções em colunas	07	Sim	<code>.apply()</code> com lambda e com função nomeada
Matplotlib — gráfico de linha	08	Sim	Receita por mês
Matplotlib/Seaborn — gráfico de barras	08	Sim	Top 5 produtos ou categorias
Matplotlib/Seaborn — gráfico de dispersão	08	Sim	Quantidade × receita por transação
Customização de visualizações	08	Sim	Títulos, rótulos, legenda, paleta e <code>figsize</code>
Subplots e layouts	08	Sim	Figura 2×2 com <code>plt.subplots()</code> e <code>tight_layout()</code>
Exportação de gráficos (.png)	08	Sim	<code>plt.savefig()</code> com <code>dpi ≥ 100</code>
Funções para transformações reutilizáveis	08	Sim	Fluxo inteiro modularizado em funções
Classes — construtor, atributos e métodos	08	Sim	<code>AnalizadorDeVendas</code> com <code>__init__</code> e <code>self</code>
Herança e <code>super()</code>	—	Não	Bônus B01 — conteúdo além do Módulo 01
Imputação de nulos, outliers, normalização	09–11	Não	Não cobrado neste mini-projeto
Estatística inferencial e correlação	12	Não	Não cobrado neste mini-projeto
Modelos preditivos (regressão etc.)	13	Não	Não cobrado neste mini-projeto

4.5. ORGANIZAÇÃO DO KANBAN

O estudante ou a squad deverá organizar o trabalho em um quadro Kanban.

Colunas obrigatórias: Backlog · A Fazer · Em Andamento · Concluído.

Ferramentas aceitas: Trello, GitHub Projects, Notion, planilha, quadro em Markdown no repositório ou imagem do quadro.

Tarefas sugeridas

- Criar o repositório no GitHub e o arquivo `salesinsight.py`
- Criar o `README.md` inicial
- Implementar a geração ou o carregamento do dataset CSV
- Implementar a função de inspeção dos dados
- Implementar a limpeza (nulos, datas inválidas, espaços, regex nos nomes)
- Implementar as colunas derivadas com `np.select` e `datetime`
- Implementar as agregações com `groupby`
- Implementar a segmentação de clientes com `lambda`
- Implementar as operações NumPy (vetorização e broadcasting)
- Criar a função de ordem superior `processar_coluna()`
- Criar o gráfico de linha (receita por mês)
- Criar o gráfico de barras (top produtos)
- Criar o gráfico de dispersão (quantidade × receita)
- Criar o painel com subplots 2×2
- Exportar as figuras em PNG
- Implementar a exportação em CSV e JSON
- Criar a classe `AnalizadorDeVendas`
- Criar a função `main()` e o bloco `if __name__ == "__main__":`
- Testar o fluxo completo no Colab ou no VS Code
- Finalizar o README com instruções e conceitos aplicados
- Organizar branches e commits no GitHub
- Gravar o vídeo de demonstração e enviar os links no AVA

4.6. VERSIONAMENTO COM GITHUB

O projeto deverá estar em um repositório **público** no GitHub.

Branches mínimas

Projeto individual:

```
main
develop
feat/pipeline-dados
docs/readme
```

Projeto em squad (até 03 pessoas):

```
main
develop
feat/leitura-limpeza
feat/analise-metricas
feat/visualizacoes
feat/classe-analisador
docs/readme-video
```

Commits mínimos

- Mini-projeto individual: **mínimo de 5 commits**
- Mini-projeto em squad: **mínimo de 8 commits**

Exemplos de mensagens (padrão conventional commits)

```
feat: cria estrutura inicial do projeto e salesinsight.py
feat: adiciona geracao e leitura do dataset de vendas
feat: implementa limpeza de dados com datetime e regex
feat: adiciona colunas derivadas com np.select
feat: implementa metricas agregadas com groupby
feat: adiciona operacoes vetorizadas com numpy
feat: cria graficos de linha, barra e dispersao
feat: adiciona painel com subplots e exportacao em png
feat: cria classe AnalisadorDeVendas para organizar o fluxo
feat: implementa exportacao de resultados em csv e json
fix: corrige conversao de datas invalidas na limpeza
docs: atualiza readme com instrucoes e conceitos
```

4.7. GRAVAÇÃO DE VÍDEO

Além do desenvolvimento, você deverá gravar um vídeo de **no máximo 5 minutos** (um por grupo, no caso de squads), abordando:

- Qual o objetivo do projeto? Demonstre o fluxo rodando do início ao fim.
- O que é necessário instalar ou configurar para executá-lo?
- Como você organizou as tarefas antes de começar a desenvolver (Kanban)?
- Quais branches você criou e qual o objetivo de cada uma?
- Explique **uma decisão técnica** do seu código — por exemplo, por que remover em vez de manter registros inválidos, ou por que usar `np.select` em vez de um laço.
- O que poderia ser melhorado no seu código?

Você poderá gravar na vertical ou na horizontal. É importante que apareça seu rosto, que o ambiente tenha boa iluminação e que sua voz seja audível com clareza. Publique o vídeo em uma pasta do Google Drive com permissão de leitura para qualquer pessoa com o link (ou no YouTube como vídeo não listado) e compartilhe o link na submissão do AVA. Recomenda-se também inserir o link no `README.md`. É obrigatório que qualquer pessoa consiga acessar o vídeo pelo link fornecido.

5. CRITÉRIOS DE AVALIAÇÃO

O mini-projeto vale de **0 (zero) a 10 (dez)** e corresponde à **Avaliação M1.1 — 30% da nota do Módulo 01**.

A coluna **Parcial** indica a pontuação atribuída quando o critério é atendido de forma incompleta; a coluna **Máximo**, quando é atendido integralmente.

Será atribuída nota 0 (zero) a projetos que apresentarem plágio de soluções encontradas na internet ou de colegas. Você está livre para consultar documentação, materiais e ferramentas de apoio, desde que adapte a solução ao desafio proposto e seja capaz de explicar, com segurança, todo o código entregue.

Apresentação do Projeto — 4,00 pontos

N.º	Critério de Avaliação	Não atingiu	Parcial	Máximo
1	Estruturou o README.md contendo obrigatoriamente: objetivo do projeto, instruções de instalação e execução, lista dos conceitos de Python e análise de dados aplicados e link do vídeo de demonstração.	0,0	0,50	1,00
2	Entregou o repositório no GitHub com branches descritivas por funcionalidade e um histórico de commits que reflete a evolução lógica do trabalho.	0,0	0,50	1,00
3	Organizou o desenvolvimento em quadro Kanban (Trello, GitHub Projects ou similar), mantendo correlação direta entre as tarefas listadas e o código implementado.	0,0	0,50	1,00
4	Demonstrou fluidez técnica ao explicar, em vídeo, o funcionamento do fluxo, ao menos uma decisão de implementação e as oportunidades de melhoria identificadas.	0,0	0,50	1,00

Desenvolvimento do Projeto — 6,00 pontos

N. º	Critério de Avaliação	Não atingiu	Parcial	Máximo
5	Carregou, inspecionou e limpou corretamente o dataset — tratando nulos, datas inválidas e ruído em textos, com uso de <code>datetime</code> e expressões regulares — e apresentou o relatório de limpeza. (RF01–RF03)	0,0	0,50	1,00
6	Criou colunas derivadas, aplicou transformação condicional com <code>np.select</code> , calculou métricas agregadas com <code>groupby</code> e segmentou clientes utilizando função <code>lambda</code> . (RF04–RF06)	0,0	0,75	1,50
7	Usou NumPy para operações vetorizadas e broadcasting, com agregações calculadas sem laços sobre arrays. (RF07)	0,0	0,50	1,00
8	Gerou as quatro figuras exigidas — linha, barra, dispersão e painel com subplots — customizadas (título, rótulos, legenda, paleta, layout) e exportadas em PNG. (RF08)	0,0	0,75	1,50
9	Organizou o código em funções reutilizáveis, implementou função que recebe outra função como argumento e criou a classe <code>AnalizadorDeVendas</code> com construtor, atributos e métodos usando <code>self</code> . (RF09)	0,0	0,25	0,50
10	Exportou os resultados em CSV e JSON, releu o JSON gravado para conferência e executou o fluxo completo sem erros a partir do bloco <code>main()</code> . (RF10–RF11)	0,0	0,25	0,50

Total da avaliação	10,00
--------------------	-------

6. CHECKLIST FINAL DE ENTREGA

Antes de enviar no AVA, confira cada item:

Estrutura e organização

- ☐ Criei o repositório **público** no GitHub
- ☐ Criei o arquivo `salesinsight.py` (e/ou o notebook)
- ☐ Criei o `README.md` com objetivo, execução, conceitos e link do vídeo
- ☐ Criei o quadro Kanban com as colunas obrigatórias
- ☐ Gerei ou incluí o dataset `vendas.csv`

Dados e transformações

- [] Inspecionei os dados com `.shape`, `.dtypes`, `.isnull()` e `.head()`
- [] Limpei nulos, datas inválidas e espaços extras, e exibi o relatório de limpeza
- [] Usei `datetime` para extrair mês, trimestre e ano
- [] Usei expressões regulares (`re.sub()` e `re.compile()`) na padronização dos textos
- [] Criei as colunas derivadas (`receita_total`, `mes`, `trimestre`, `ano`, `faixa_receita_item`)
- [] Usei `np.select` para a transformação condicional vetorizada
- [] Usei `groupby` para agregar por mês, produto, categoria e região
- [] Segmentei os clientes em Bronze/Prata/Ouro usando função `lambda`

NumPy

- [] Converti uma coluna para array com `.to_numpy()`
- [] Usei ao menos três funções de agregação do NumPy
- [] Demonstrei broadcasting e filtragem booleana, sem laços sobre o array

Visualização (foco da Semana 08)

- [] Gerei o gráfico de **linha** (receita por mês)
- [] Gerei o gráfico de **barras** (top produtos ou categorias)
- [] Gerei o gráfico de **dispersão** (quantidade × receita)
- [] Gerei o **painel com subplots 2×2** e `fig.suptitle()`
- [] Customizei todas as figuras (título, rótulos, legenda, paleta, `figsize`)
- [] Exportei todas as figuras em PNG com `plt.savefig()` e `tight_layout()`

Código e arquivos

- [] Organizei o fluxo em funções com parâmetros, retorno e docstring
- [] Usei funções `lambda` em pelo menos 2 contextos distintos
- [] Criei uma função que recebe outra função como argumento
- [] Criei a classe `AnalizadorDeVendas` com `__init__`, atributos e métodos com `self`
- [] Exportei resultados com `.to_csv()` e `json.dump()`
- [] Li o JSON de volta com `json.load()` para conferência

- [] Criei a função `main()` e o bloco `if __name__ == "__main__":`
- [] Executei o projeto do zero e ele rodou sem erros

Entrega

- [] Fiz commits descritivos e usei branches com nomes claros
- [] Gravei o vídeo de até 5 minutos
- [] Publiquei o vídeo com permissão de acesso por link
- [] Enviei os links do repositório, do Kanban e do vídeo no AVA antes do prazo

7. MODELO DE README.md (SUGESTÃO)

Use o modelo abaixo como ponto de partida, adaptando-o ao seu projeto:

```
# SalesInsight PY

## Sobre o projeto
Analise e visualizacao de dados de vendas desenvolvida em Python. O
projeto carrega, limpa, transforma, agrega e visualiza um dataset de
vendas, gerando metricas por periodo, produto, categoria e regioao,
além de uma segmentacao simples de clientes por faixa de gasto.

## O que o projeto analisa
- Receita total e volume de vendas por mes e por trimestre
- Top produtos e categorias por receita
- Desempenho por regioao
- Segmentacao de clientes por nivel de gasto (Bronze, Prata, Ouro)
- Relacao entre quantidade vendida e receita por transacao
- Exportacao de relatorios em CSV e JSON e de graficos em PNG

## Conceitos aplicados (Modulo 01 - Semanas 01 a 08)
- Logica de programacao: variaveis, tipos, operadores, condicionais
- Estruturas de dados: listas, tuplas, dicionarios e compostas
- Funcoes: parametros, retorno, docstrings, lambda, ordem superior
- Leitura e escrita de arquivos CSV e JSON
- Modulo datetime e expressoes regulares (re)
- Pandas: Series, DataFrames, filtros, groupby, transformacoes
- NumPy: arrays, operacoes vetorizadas e broadcasting
- Matplotlib e Seaborn: linha, barra, dispersao, subplots, export
- Introducao a classes: construtor, atributos e metodos
- Git e GitHub: branches, commits e GitFlow simplificado

## Como executar

### Google Colab (recomendado)
1. Faça upload de salesinsight.py e de vendas.csv para o Colab.
2. Execute: !python salesinsight.py
3. Ou cole o conteudo em celulas de um notebook .ipynb

### Localmente com VS Code
```



```
1. Instale o Python 3.10+ e o VS Code.
2. Instale as dependencias:
   pip install pandas numpy matplotlib seaborn
3. Execute no terminal: python salesinsight.py

## Estrutura do projeto
salesinsight-py/
|-- salesinsight.py          # fluxo principal
|-- vendas.csv              # dataset (gerado ou externo)
|-- README.md
|-- outputs/
    |-- metricas_por_mes.csv
    |-- segmentacao_clientes.csv
    |-- estatisticas_gerais.json
    |-- graficos/*.png

## Decisoes tecnicas
Explique aqui, em poucas linhas, ao menos uma escolha que voce fez -
por exemplo, por que remover os registros invalidos em vez de
mante-los, ou por que usar np.select em lugar de um laço for.

## Ferramentas utilizadas
- Python 3.10+
- Google Colab / VS Code
- Bibliotecas: pandas, numpy, matplotlib, seaborn, re, json,
  datetime, os, random
- GitHub (e GitHub Desktop, opcional) para versionamento
- Trello / GitHub Projects para o Kanban

## Video de demonstracao
[inserir o link do Google Drive ou do YouTube aqui]
```

Documento elaborado para o curso Desenvolvimento de IA para Análise Preditiva — Módulo 01, Semana 08.